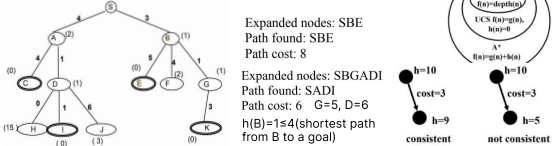


(Blind) Search Strategies (states, operators, goal state, path cost)
Complete: guaranteed to find a goal node if one exists. Optimal: guaranteed to find a least-cost solution.
b = max branching factor (assume is finite); d = depth of the optimal solution (assume is finite); m = maximum depth of the state space; ε = minimum step cost (for UCS/C*)
BFS: Complete if finite b, Optimal if same step cost, O(b*b*d) time - fast, O(b*b*d) space - high, Pro: Guarantees shortest path, simple to implement; Con: High memory usage, slow for deep problems
UCS: Complete if costs>0, Optimal, O(b*(C*/ε)) - fast, O(b*(C*/ε)) - high, Finds optimal solution with variable costs; Can be slow with many small-cost steps, high memory requirements
DFS: Complete (if finite d), Not optimal, O(b*m)-lowest, O(b/m)-low, Low memory usage, can reach deep solutions quickly; Can get stuck in deep paths, not guaranteed to find the best
DLS: Complete (if finite d = 1). Not optimal. O(b*-fast, but depends on l, O(b-l)-low like DFS; Must choose a depth limit l in advance (requires prior knowledge to set); **DLS** → **DFS**: l = m
IDS: Complete if finite b, Optimal if same step cost (so that total cost of a path is exactly its depth, the shallowest goal is also the cheapest.), finds the shallowest goal (like BFS) without getting stuck down infinite paths. O(b*b*d)-fast, O(b/d) - low (like DFS). Combines benefits of BFS and DFS. Cons: Requires re-expanding nodes multiple times, increasing computational overhead.



g(n) = cost so far to reach n (edge)(UCS minimize it)
h(n) = estimated cost from n to the goal (node)(GS minimize it)
f(n) = estimated total cost of path through n to the goal; f(n) = g(n) + h(n). (A* minimize it)
Informed (Heuristic) Search Strategies - Use extra knowledge (heuristics) to guide search, h must estimate the cost at a goal node to be 0;
GS: Complete (if finite d), Not optimal, O(b*m), O(b*m), Keeps every node in the memory. A good heuristic can improve time; → UCS: h(n) = g(n)
A*: the best optimal algorithm using g if h is consistent
An **admissible** heuristic never overestimates the true distance to a goal: $\forall n, h(n) \leq h^*(n)$
A **consistent** heuristic satisfies the triangle equality: $\text{Vedge}(u \rightarrow v), h(u) \leq c(u, v) + h(v)$
↔ $f(n) = g + h$ does not decrease along any path
• Consistent ⇒ admissible | Admissible ≠ consistent
• If h(n) is admissible, A* is optimal; if h(n) is non-admissible, A* is not optimal but could still find a good answer in a reasonable time, depending on how good the heuristic is.
• If h(n) is consistent, A* is optimally efficient, A* will expand less or equal nodes than any other optimal algorithm using h(n); if h(n) is not consistent, A* may end up expanding more nodes and even re-expand nodes whose f-values later decrease.
Dominant heuristic (may need a lot of time to compute):
• Given admissible heuristics, h2 dominates h1, if for all nodes n, h1(n) ≤ h2(n).
• A* with a dh will expand fewer nodes ⇒ more efficient since it is closer to the real cost.

Relaxed problem: removing some constraints, making it easier to solve (e.g. swap any 2 variables). The solution of relaxed version of problem serves as a admissible heuristic h(n) for the original problem.

Local Search / Optimization Methods - not complete and not optimal, but use very low memory. They optimize for the node's score instead of the path length to a goal node.
Hill-Climbing Algorithm - good choice for hard, practical problems.
Pros: Uses very little memory, Finds reasonable solutions in large spaces where systematic algorithms are not useful; **Cons:** Can easily get stuck in a local optimum
Escaping Bad Local Optima
• Random restart: take the best result over k iterations with different starting states.
• Plateaux (flat areas): allow equal v moves - allows a node to only visit k number of times (avoid infinite visit loop), and break ties by choosing smaller visited times.
• Ridges (when single-step lowers v, but a combination of steps would climb): 1. Macro move - combine two or more actions into one move that directly increases v. 2. Limited lookahead - explore sequences of a few moves ahead and pick the sequence that eventually leads upward.
Beam Search Pros: balances breadth & memory. **Cons:** not complete or optimal.
vs. Hill-Climbing: - Stopping Criteria: Hill climbing - if h(current) ≥ h(next), stop.
• 1 initial state: Beam Search - at each step keeps k best nodes.
• K random initial states: Beam Search - k beams run in parallel, passing useful information.

Simulated Annealing Pros: Escapes local optima by accepting bad moves early. Easy to implement. **Cons:** Choice of cooling schedule is tricky; can be slow if you cool too slowly, or poor-quality if you cool too fast. $P(\text{accept worse neighbor}) = e^{-\frac{p(\text{new}) - v(\text{old})}{T}}$
• Starts at a randomly selected initial state.
• Repeatedly choose a random adjacent state: Accept if the value of the new state is better. Otherwise, assign probability p, where T is the temperature constant
Theorem: If the schedule lowers T slowly enough, the algorithm will find global optimum.
• Terminate after T is very low (almost zero), or you've done enough iterations. At that point, you're unlikely to accept any worse moves.

Cooling too fast → T falls too quickly → easily trapped in local optima
Cooling too slow → T stays high for too long → over-exploration, low efficiency.
Genetic Algorithms - selection, crossover, mutation (expect average fitness of the population increases as worst individual is forgotten). **Pros:** Easy to implement. Combine uphill tendency (based on the fitness function) & random exploration (based on crossover and mutation). **Cons:** Success depends on the representation (encoding).
Taxonomy of Clustering Algorithms
• **Partitional:** Split into k (or an unspecified number of) clusters.
• **Hierarchical:** Create a nested set of clusters. 2 versions: **Agglomerative and Divisive**.
K-Means Clustering Example: epoch1 - start: → clusters: K1={3, 5}, K2={7, 10, 13}
3: seed of cluster K1 → 3eK1
5: d(c1={3, 5})=2, d(c2={10, 5})=5 → 5eK1
7: d(c1={3, 7})=4, d(c2={10, 7})=3 → 7eK2
10: seed of cluster K2 → 10eK2
13: d(c1={3, 13})=10, d(c2={10, 13})=3 → 13eK2
new centroids:
c1_new = (3+5)/2 = 4
c2_new = (7+10+13)/3 = 10
only iterate through each datapoint once!

Games - Determinism vs. non-determinism; Perfect vs. imperfect information; Zero-sum (adversarial) vs. non-adversarial
In 2-player, 0-sum games, with perfect information:
• **Minimax:** Complete (if finite d), Optimal, same with DFS
• **Alpha-Beta Pruning:** Complete (if finite d), Optimal, O(b^d) - worst case, O(b^(d/2)) - best.
alpha cut off: child's b ≤ parent's a; beta cut off: parent's b ≤ child's a
• It is not visible to consider the whole game tree, so we need to cut the search off and apply evaluation function which estimates the goodness of a state: linear weighted sum
• Games by chance can be handled by **expectiminimax** that evaluates chance nodes by taking the average utility of all children, weighted by the probability of each child
Possible improvements to Game-Playing: The search depth can be extended locally to explore these states; An IDS can be used instead of a DFS when there is a strict time requirement to guarantee that a decision is made; Trivial moves or opening moves can be hardcoded; Use heuristic pruning to improve the branching factor. Order move exploration based on heuristic score to improve the effectiveness of alpha-beta pruning.

Basic ML
Supervised: Training data is provided with expected output.
Unsupervised: Training data has no expected output.
Reinforcement: Training error is evaluated based on a heuristic.
Associations: Training data is a database.
Tasks: Classification: Assign a label. Regression: Produce a continuous value. Clustering: Assign an abstract group (unsupervised).
k-Nearest Neighbor (k-NN): uses the k closest neighbors and takes a majority vote (for classification) or average (for regression).
• **Nominal fields:** Distance is 0 if matching, 1 otherwise;
• **Numeric fields:** Normalize between 0 and 1;
• **Missing fields:** Assign worst-case distance.
• **Weighted k-NN** gives closer neighbors more influence (w=1/d) ⇒ often the effect of outliers.
• Training time: Fast, Classification time: Slow (O(mn)) for m training examples, n attributes). Memory usage: High (O(mn))
Pro: Simple method; works well in practice (like 1R and Naive Bayes)
Cons: Query-time cost: must compare to all m training points ⇒ efficient indexing
• **Curse of dimensionality** (points all look equally far apart) - "nearness" is ineffective in high dimensions ⇒ feature selection to reduce dimensionality
• **Parameter sensitivity:** k too small ⇒ overfit, noisy decisions k too large ⇒ overly smooth, biased boundaries ⇒ choose k ≤ sqrt(m) for m examples.
• **Distance domination:** one feature with large range can drown out others ⇒ **Normalize** numeric features to [0,1]. Ensure attributes in different scales have the equal contribution to the model
• **Irrelevant/noisy attributes:** worsen "nearness" ⇒ Weight features by relevance, or perform feature selection.

Lazy vs Eager Learning - Lazy Learning (k-NN): No model is built in advance, computation occurs during classification. requires full training data;
Eager learning (DT, Naive Bayes, SVMs): build a model ahead of time, then classify quickly.
uses rules to test the values of a single attribute.
1-R Algorithm - chooses final rule based on lowest training-error attribute.
Pro: Simple and computationally cheap algorithm e.g. 2no, 3yes → yes, errors: 2/5
• **Dealing with Missing Fields:**
1. Ignore instances with missing attributes. 2. treat the missing value as another possible value of the attribute; this assumes that the absence of a value is significant.
3. Fill missing fields with:
• a most common value for attribute A in all training examples;
• a most common value for attribute in all training examples from the same class as the class of the example with the missing value
Manhattan distance: $|x_1 - x_2| + |y_1 - y_2|$
Euclidean distance - most frequently used: $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

Minkowski distance - generalization of Euclidean and Manhattan:
 $D(A, B) = \sqrt[q]{|x_1 - x_2|^q + |y_1 - y_2|^q + \dots + |x_n - x_n|^q}$ q = positive integer
Can be viewed as an optimization problem: find k clusters that minimize the **squared error (SSE)** (Not guaranteed to find the global minimum)
Pros: 1.Simple and very popular; 2.Relatively efficient (Modest space complexity, high time complexity) (Fast convergence (typically) - small number of iterations). 3.Not sensitive to the order of input examples
Cons: 1.Not guaranteed to find the optimal solution. 2.Sensitive to initialization - In practice: run it several times with different initialization, select the best clustering found
3.Does not work well for clusters with non-spherical shape, non-convex shape. 4.Does not work well with data containing outliers (which can drag the centroid far away) → Pre-processing is needed - outlier detection and removal) 5.The best number of clusters k is not known in advance. 6.Interpreting the semantic meaning of each cluster is difficult.
Image Compression Example: For k=10 and 2+2 blocks, 1MB=1024*1024bytes, 512*512 input vectors for clustering, k centroids with dimensionality 4 = k*4*8 bits
The index of the closest centroid will replace its values: log2k bits per block
Compress rate(compressed/original) = $\frac{k*4*8 + 512*log_2(k)}{1024*1024*8}$ bits = 0.239 for k=200, 0.063 for k=4
codebook storage, indices for each vector
Nearest Neighbor Clustering Algorithm threshold t = 3
A: K1={A}.
B: d(A,B)=1 < t ⇒ K1={A,B}.
C: d(C,A)=4, d(C,B)=2; d(C,B)=dmin < t ⇒ K1={A,B,C}.
D: d(D,A)=5, d(D,B)=6, d(D,C)=4; d(D,C)=dmin < t ⇒ K2={B,C,D}.
Final clusters: K1={A,B,C}, K2={D}

Hierarchical Clustering Algorithms
1. Agglomerative Clustering(bottom-up) - merges clusters iteratively
Single Link vs. Complete Link Algorithm
Single link (MIN) - Cluster distance = minimum distance between any two points
• Merges whenever any pair is close ⇒ suffers "chain effect" (one from each cluster)
• May connect unrelated points via chains of nearby neighbors.
• More sensitive to noise and outliers
Complete link (MAX) - Cluster distance = maximum distance between any two points
• Only merges when all points are sufficiently close
⇒ generates more compact clusters
• Less sensitive to noise and outliers
2. Divisive Clustering(top-down) - splits a cluster iteratively: Place all items in one cluster; iteratively split clusters in two until all items are in their own cluster
• Splitting is based on the distance between clusters - split if the distance ≥ threshold d

Dealing with Numeric Fields:
1. Discretize the field into ranges with breakpoints. → May Overfitting (see next page)
2. Better way: Aggregate adjacent attributes with the same majority class into attributes.
Bayesian Classifiers:
• Use the **Naïve-Bayes assumptions:** (Why unrealistic?): Assume:
1. Attributes are **conditionally independent** of each other;
2. Attributes are **equally important** for the classification
$$P(H|E) = \frac{P(H \cap E)}{P(E)} = P(E|H) \frac{P(H)}{P(E)} \frac{P(\text{yes} | E) = \frac{P(\text{yes} | \text{yes}) P(\text{yes})}{P(\text{no} | \text{yes})} = \frac{2 + m_{\text{pi}}}{9 + m}$$

e.x. used for predict class
• **Dealing with Zero Probabilities:**
• **Laplace Correction:** Add 1 to numerator, k to denominator (k = number of attribute values), Ensures no probability is ever 0. This will cause minimal impact when the dataset is large.
• **m-estimate:** n_i = count of x_i in training data, p_i = prior estimate for x_i (e.g., 1/K), m = smoothing parameter, Laplace is a special case with m=k and p_i = 1/k
• **Dealing with Missing Values:**
• Missing attribute value in the new example: do not include this attribute
• Missing value in a training example: do not include this value in the counts (denominator)
• **Dealing with Numeric Attributes:**
• Use a **probability density function** assuming normal distribution. P(E|yes) change to: x = attribute value of this new example; Mean value μ, standard deviation σ

$$F(x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad \text{e.g. } f(x) = f(\text{temp}=60|\text{yes})$$
$$\mu = \frac{1}{n} \sum_{i=1}^n x_i \quad \sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2}$$

Pros:
• Simple approach, probabilities are easily computed with independence assumption
• **Clear semantics for representing, using and learning probabilistic knowledge**
• Efficient: one scan of data O(p*k), where p = examples, k = attribute values
• Robust to noise, noises are averaged when estimating the conditional probabilities
Cons: • **Assumes conditional independence** (often violated); Solution: apply feature selection beforehand to identify and discard correlated (redundant) attributes
• **Assumes normality for numeric features** (often invalid); Solution: 1. Discretize the data first, i.e. numerical → nominal attributes. 2. Use other probability density. 3.Transform the attribute using a suitable transformation into a normally distributed one (sometimes possible). 4. Use kernel density estimation which does not assume any particular distribution
Evaluating Classifiers - Generally: The larger the training data, the better the classifier, The larger the test data, the better the accuracy estimate.

examples	# assigned to class yes	# assigned to class no
# from class yes	true positives (tp)	false negatives (fn)
# from class no	false positives (fp)	true negatives (tn)

Accuracy: Correct / (TP + TN) Total
Error rate: Incorrect / Total = 1 - Accuracy
Precision: TP / (TP + FP)
Recall: TP / (TP + FN)
F1 score: 2 * P * R / (P + R)
When might you prefer F, or accuracy?
1. classes are imbalanced (P ≠ N) 2. when FP and FN both matter.

Single holdout (train/test split). Problem: the examples in the training set might not be representative of all classes, especially rare classes → Stratification
Stratification: Split data so that each class is represented with approximately equal proportions in both data sets (training and test) **Problem:** Repeated holdout (multiple random splits), can be improved by ensuring that the test sets do not overlap → Cro Vali
Cross Validation (k-fold): Avoids overlap in testing sets. Split data into k different sets of similar size. Build k classifiers trained on k-1 of the sets, with one set being held out as the testing set. Every subset will be the testing set once. Average the statistics between folds.
[P1 L1 Simulated A] T → ∞ and constant: P ≈ 1; Almost every uphill move is accepted, so the algorithm performs a nearly random walk and fails to converge toward better solution.
KN, DT, NB, SVM - Comparison Dim / Large Dataset / Frequent Efficient Updates
KN - Poor (curse of dimensionality) / Poor (O(N*d) per query; heavy storage) / Trivial (just append new points) **DT** - Moderate (sparse splits; overfitting) / Reasonable (O(N*d*log N) batch train) / Hard (must revisit/rebuild splits) **NB** - Excellent (O(N*d) training; independence) / Excellent (one-pass, linear in N-d) / Very easy (adjust counts in O(d), but relies on a strong independence assumption.) **SVM** - Good (kernel trick handles implicit high-D) / Poor (O(N^2-N^3) training; high memory) / Difficult (full retrain)
[P2 L1 Bad-PP] **Validation Set:** Available data is divided into 3 subsets: 1) Training set - used for computing the gradient and updating the weights. 2) Validation set - used for early stopping (when the error on the validation set increases for a pre-specified number of iterations.). 3) Test set: compare different algorithms after training.

Single Link E.x.
Level 0: (0, 4), (A), (B), (C), (D)
Level 1: (1, 3), (A,B), (C), (D)
Level 2: (2, 2), (A,B,C), (D)
Level 3: (3, 1, (A,B,C,D))
Complete Link E.x.
Level 0: (0, 4), (A), (B), (C), (D)
Level 1: (1, 3), (A,B), (C), (D)
Level 2: (2, 3), (A,B), (C), (D)
Level 3: (3, 2), (A,B), (C,D)
Level 4: no merging
Level 5: no merging
Level 6: merge
Initially d is set to a big value that is decremented at each level.
Summary:
Pros: Suitable for tasks with natural nesting relationships between clusters (taxonomies, hierarchies)
Cons: 1. Computationally expensive which limits its applicability to high dimensional data. 2. Not incremental - assumes all data is present.
Dendrogram:
A B C D A B C D
$$\text{splitInformation}(S|A) = -\sum_{i=1}^n \frac{|S_i|}{n} \log_2 \frac{|S_i|}{|S|} \quad \text{splitInformation}(S| \text{Idcode}) = -\frac{1}{n} \log_2 \frac{1}{n} * 14 = 3.807 \text{ bits}$$
$$\text{GainRatio}(S|A) = \frac{\text{Gain}(S|A)}{\text{splitInformation}(S|A)} \quad \text{GainRatio}(S| \text{IdCode}) = \frac{0.940}{3.807} = 0.246 \text{ bits}$$

Leave-One-Out Cross-Validation (LOOCV): Special case of k-fold where k = n, and n is the total number of examples.
Pros: 1. The greatest possible amount of data is used for training ⇒ increases the chance for building an accurate classifier. 2. Deterministic procedure - no random sampling is involved (no point in repeating the procedure - the same results will be obtained) **Cons:** 1. high computational cost ⇒ useful for small data sets
Comparing Classifiers - Paired t-test
• Null hypothesis: The two classifiers have the same mean accuracy.
• Use the same train/test splits (e.g., k-fold). Calculate differences d, sd σ, confidence interval Z. Check if the difference is random chance. Use paired t-test on the k paired accuracies to see if the difference is statistically significant. If it is, conclude: "Yes, C1 is significantly better than C2 at 95% confidence."
Cost-Sensitive Evaluation: Not all misclassification errors are equal. Solution: Adjust error weights accordingly (e.g., misclassifying non-spam as spam = 10x penalty), calculate weighted A, P, F1.
Decision Trees
Tree Construction Stop if 1) All examples have the same class. → assign such class 2) All examples have identical attribute values but differing classes (label noise) → assign majority class. 3) The subset is empty → assign majority class from the parent set.
Example: It is not possible to create leaf nodes as the resulting partitions of the dataset are not "pure" - they contain examples from different classes. Hence, both child nodes need to further expanded by selecting attribute to split on.

Entropy measures the impurity or unpredictability of a dataset. log 0 is not defined, but assume log0=0.
Range of the Entropy for Binary Classification: H(S) in [0,1]
$$H(S) := -\sum_i P_i \log_2 P_i$$

• H(S)=0 ⇒ all examples in S belong to the same class (no disorder, min entropy)
• H(S)=1 ⇒ equal number of yes & no (S as disordered as it can be; max entropy)
• Low entropy: the values of S are more predictable → skewed distribution
• High entropy: the values of S are all over place → flat (uniform) distribution
Information Gain measures the expected reduction in entropy due to sorting on an attribute. It compares the entropy of the dataset before and after the split.
H(S) is the entropy before splitting A **Gain(S|A) = H(S) - ∑ P(v_i) H(S|v_i)**
Information Gain of an attribute A on dataset S is how much the entropy decreases if you split S by A. e.g. H(S)-H(S|A)
Second term is the expected entropy after split, aka remainder or conditional entropy
Example: For Syes and Sno, Parent Entropy T1=H(S)=(9/14, 5/14) = 0.940 bits.
• **Outlook = Sunny:** 5 ex (2 "yes", 3 "no") → entropy=(2/5, 3/5) = 0.971 bits.
• **Outlook = Overcast:** 4 ex (4 "yes", 0 "no") → entropy = 0 bits (pure).
• **Outlook = Rain:** 5 ex (3 "yes", 2 "no") → entropy - 0.971 bits.
• **Weighted sum T2=H(S|Outlook)=(5/14) * 0.971 + (4/14) * 0 + (5/14) * 0.971 = 0.693 bits.**
Information Gain: 0.940 - 0.639 = 0.247 bits.
The best attribute is the one with highest information gain
• It reduces most the entropy of the parent node
• It is expected to lead to pure partitions (entropy = 0) fastest
Overfitting: the error on training data is very small but the error on new data (test data) is high. **Reasons:** 1. Training data is too small ⇒ not enough representative examples to build a model that can generalize well on new data.
2. Noise in training data (incorrectly classified examples) ⇒ DT learns them but they're incorrect ⇒ will incorrectly classify new examples. **How to avoid?**

1. **Pre-Pruning:** Stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data. Using e.g. Minimum number of instances per leaf node, Maximum tree depth. Minimum gain threshold. Goal: prevent the tree from modeling noise or overly small partitions. Con: may stop too early and underfitting (must decide a good stopping rule)
2. **Post-Pruning by Subtree Replacement:** Grows the full tree, then **prune** it. Uses a separate **validation set** (distinct from training and test sets) to assess pruning impact. If replacing a subtree with a leaf improves or with same validation accuracy, perform pruning. Iterate until no further improvement is possible. **Pros:** reduces tree size and avoids memorizing noise.
3. **Post-Pruning by Rule pruning:** Convert a complete DT to a set of rules, then **prune** it. Iteratively prune rules by removing preconditions that are not harmful. Sort the pruned rules by their validation-set accuracy and use that ordered rule list for classification of new instances. **Pros:** Rule sets can be more readable than large trees.

Handling Numeric(continuous) Attributes - Discretization
1.Sort the training examples by the numeric attribute. 2.Find all adjacent pairs where the class label changes (e.g. one is yes, the next is no). 3.For each such adjacent boundary, consider a candidate split point halfway between the two numeric values. 4.Evaluate the information gain at each candidate threshold. 5.Choose the threshold that yields the highest gain.
Dealing with Missing Values (e.g. At node n: X with a missing value for wind and 10 other examples) 1.Assign probability to each branch using the examples without missing values that have reached. 2.Use these probabilities in Gain for the examples with missing values.
Handling attributes with different costs: we want DT to branch on attributes with lower cost first, so we can pay low cost when classifying new examples
⇒ Incorporate the cost in Gain by penalizing attributes with high cost:
w=0: ignore cost entirely w=1: fully apply penalty
Alternate Measures for selecting attributes **Tan & Schlimmer** Nunez, where w ∈ [0,1] determines cost importance
$$\text{Gain}^*(S|A) = \frac{\text{Gain}(S|A)}{\text{Cost}(A)}$$
$$\frac{2^{\text{Gain}(S|A)} - 1}{(\text{Cost}(A) + 1)^w}$$

• Reason: highly-branching attributes (ID number) are more likely to create pure subsets → low entropy → high information gain
• **overfitting**
• **Gain Ratio** reduces Gain's bias towards highly branching attributes.
1.Takes number and size of branches into account when choosing an attribute.
2.Penalizes highly-branching attributes by incorporating **SplitInformation**, which is the entropy of S with respect to A.

2) Phases: Growing the tree: hill climbing guided by information gain; uses training data set. Pruning the tree: various approaches. (e.g. sub-tree replacement + validation set to decide how much to prune - performs a search similar to greedy search to choose the best sub-tree to be replaced with a leaf node)

Pros: Easy to implement, Efficient, Interpretable: The output of the DT (the tree = a set of rules) is considered easier to understand by humans and use for decision making than the output of other algorithms, e.g. NNs and SVMs (if the tree is not too big)

Cons: Vulnerable to overfitting.

Neural Networks

Perceptrons (Complete: Guaranteed to converge if training data is linearly separable, Not optimal: the separating hyperplane found depends on the initial weights and the order of presentation.)

Supervised perceptron use: $a = f(\mathbf{w} \cdot \mathbf{p} + b)$

Step transfer function:

Perceptrons form a linear decision boundary $\mathbf{w} \cdot \mathbf{p} + b = 0$

Solve problem with a perceptron with step transfer function without a bias?

→ if no bias, the decision boundary passes through the origin where one of the points lays.

Learning rules: Error $e = t - a$, $pT =$ input vector

Example: 1) starting point: $\mathbf{w} = [0 \ 0 \ 0]$, $b = 0$

2) Iteration 1: $p_1 = [1 \ 0 \ 0]T$, $t_1 = 1$, $a = \text{step}([0 \ 0 \ 0] [1 \ 0 \ 0] + 0) = \text{step}(0) = 1$

$e = t_1 - a = 1 - 0$, $\mathbf{w}_{\text{new}} = [0 \ 0 \ 0] + 0 [1 \ 0 \ 0] = [0 \ 0 \ 0]$, $b_{\text{new}} = 0 + 0 = 0$ 1 epoch

3) After one full pass (epoch), test all examples: All correctly classified, stop. A predefined maximum number of epochs is reached, stop; Else, start a new epoch. Final check at end of epoch not update weights.

Con: Cannot solve non-linearly separable problems (e.g. XOR); Does not support multiple classes directly. Are restricted to binary outputs; no probability estimation or confidence scores.

Multi-layer Neural Networks

A (MLP) with at least one hidden layer can represent XOR. Each hidden neuron creates a linear decision boundary. The output layer combines them, form a nonlinear boundary. Example: Given 2-layer NN with 10 input, 5 hidden, 3 output units, $\mathbf{w}$ weights: $10 \times 5 + 5 \times 3 = 65$, biases: $5 \times 1 + 3 \times 1 = 8$, total: $65 + 8 = 73$

Instead of step function, we use Differentiable Transfer Function (need to be smooth and can be differentiable everywhere):

$a = \log(\text{sgn}(f))$

$a = \frac{1}{1 + e^{-n}}$

$a = \text{tansig}(n)$

Log-Sigmoid Transfer Function

Tan-Sigmoid Transfer Function

Universal Approximation Theorem (Cybenko): Backpropagation NNs with 1 hidden layer are universal approximators - can learn any continuous function with arbitrary low error.

Cons: Only finds local minima, not always find a good solution. Sensitive to initial weights, LR, number of hidden layers and neurons. Training time can be long.

Why do we need more than 1 hidden layer?

- This is an existence theorem, i.e. it says that there is a NN with 1 hidden layer that can do this but doesn't tell us how to find this network
- This doesn't mean that 1 hidden layer is the most effective representation that will result in the fastest learning, easiest implementation or best solution (i.e. ability to classify correctly new examples).

Initialization of Weights: Uniform random in $[-1, 1]$ (or a small range around zero). This breaks symmetry, so different neurons learn different features rather than all behaving identically.

Input Neurons: Corresponding to attributes. 4 numeric attributes → 4 input neurons. 1 categorical (nominal) attribute like "Weather" = {Sunny, Overcast, Rainy}, → 3 input neurons (one-hot encoding: a way to represent categorical (non-numeric) values as binary vectors).

Output Neurons: Corresponding to class. Binary classification → 1 output neuron whose value is interpreted as "probability of class 1" (with class 0 being 1 minus that probability). k classes → 1 neuron per class and encode the target in a one-hot vector (Often 0.9 is used instead of 1, 0.1 instead of 0 to avoid flat gradients.)

Hidden Neurons & Layers: Common heuristic ≈ $(\text{num_input} + \text{num_output})/2$

Too many → overfitting; too few → underfitting (insufficient capacity). Often one tries a small network first, then gradually increases size until performance on a validation set stops improving.

Error Function: For classification, we measure how far off the network's outputs are from the desired targets. A very common choice is the **Sum of Squared Errors (SSE)**: For neuron i , d_i : target output,

$E = \frac{1}{2} \sum e_i^2 = \frac{1}{2} \sum (t_i - a_i)^2$

a_i : actual output. (Learning = minimizing error (search in the weight space))

Backpropagation (computes the gradient of the error with respect to each weight $\mathbf{w}$ in the network, by applying the chain rule and propagating the error from the output layer back toward the inputs to update weights and biases.

- Weight update
- Weight change rule
- Gradient decomposition

$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$

$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}} = \eta \delta_i a_j - \eta \frac{\partial E}{\partial w_{ji}} = \eta \delta_i a_j$

$\delta_i = (d_i - a_i) (1 - a_i^2)$

$\delta_j = (1 - a_j^2) \sum_i w_{ji} \delta_i$

Steepest Gradient Descent: Goal: Find the set of all weights (and biases) that minimize the error E . The weight vector lives in a high-dimensional space ("weight space"), and $E(\mathbf{w})$ is like a hilly landscape over that space. **Update Rule:** At each iteration, adjust $\mathbf{w}$ in the direction of steepest descent of the error, step size is controlled by learning rate η .

Cons: Not guaranteed that after each adjustment the error will be reduced if a local minimum is reached. Not optimal- is guaranteed to find a minimum but it might be a local minimum. Error space: many local minima and 1 global minimum.

Learning Rate: Too small → Convergence is extremely slow (tiny steps down the slope). Too large → may overshoot minima, causing oscillations or divergence (weights keep jumping past the minimum). Solution: adaptive LR adjusts during training (Adam). Early stopping. Try different initial config. **Overfitting:** Training examples are noisy, Small dataset not representative, Number of the free (trainable) parameters is bigger than the number of training examples. **Stopping Criteria:** 1) Training error threshold: total error is below a certain value (then be propagated again, calculate total error); 2) Max epochs: predefined number of epochs reached; 3) Early stop: validation set → prevent overfitting. **Speeding up the Convergence:** 1) Increase the learning rate (faster but unstable when falling into the steep valley contains min → overshooting the minimum

2) Momentum Term $\Delta w_{pq}(t) = \eta \delta_q o_p + \mu \Delta w_{pq}(t-1)$

iteration's update, η : learning rate, δ_q : error signal at neuron q , o_p : output of neuron p

$\mu \in [0,1]$: momentum coefficient (typically 0.6-0.9)

Example: (forward pass) - Present a training example and propagate it through the network to calculate the actual output of the network

ex.1: target output 1.0, i.e. class 1 - Activations of the hidden neurons: $\text{net3} = 0.1 \times w_{13} + 0.2 \times w_{23} + b_3 = 0.14$, $0.3 - 1/(1 + e^{-(\text{net}.3)}) = 0.53$, $\text{net4}, \text{net5}, \text{net6}$

- Activations of the output neurons: $\text{net6}, \text{o6}, \text{net7}, \text{o7}$ (backward pass) - Compute the error (the values for the output neurons) - Starting with output layer, repeat for each layer in the network - propagate the values back to the previous layer - update the weights between the two layers

Calculate the output errors δ_6 and δ_7 ($d_6 = 1$, $d_7 = 0$)

$\delta_6 = (d_6 - o_6) \times 0.6 \times (1 - o_6) = 0.12$, $\delta_7 = 0.6$

- Calculate the new weights between the hidden and output neurons ($n = 0, 1$): $\Delta w_{36} = \eta \delta_6 o_3 = 0.006$, $w_{36}^{\text{new}} = w_{36}^{\text{old}} + \Delta w_{36} = -0.394$, $\Delta w_{37} = \eta \delta_7 o_3 = 0.012$, $w_{37}^{\text{new}} = w_{37}^{\text{old}} + \Delta w_{37} = 0.012$
- Calculate the errors of the hidden neurons 63, 64, 65
- Calculate the new weights between the input and hidden neurons ($n = 0, 1$)

$\Delta w_{13}, w_{13}^{\text{new}}, w_{14}^{\text{new}}, w_{15}^{\text{new}}, w_{16}^{\text{new}}, w_{23}^{\text{new}}, w_{24}^{\text{new}}, w_{25}^{\text{new}}, b_3, b_4, b_5$

- At the end of the epoch - check if the stopping criterion is satisfied.

Pros: If previous change was large → current change will be large and vice versa. Speeds up convergence along shallow gradient valleys (convergence is slow as the direction that has to be followed has a very small gradient → oscillations). The use of momentum typically smooths out the oscillations and produces a more stable trajectory.

Summary: Backpropagation NNs are used for supervised learning (classification or regression). **Theoretically they can**

- Form arbitrary decision boundaries (i.e. both linear and non-linear)
- Are universal approximators - can approximate any function arbitrarily well

In practice: - May not always find a good solution - it will iterate until a local minimum of the error is reached. (no guarantee, unlike perceptron learning rule)

- Performance is sensitive to the starting conditions (weights initialization)
- Sensitive to the number of hidden layers and neurons
 - Too few neurons - underfitting, unable to learn what you want it to learn
 - Too many - overfitting, learns slowly
- Sensitive to the value of the learning rate
 - Too small - slow learning; Too big - instability or poor performance
 - The proper choice depends on the nature of examples
- Trial and error; Refer to the choices that have worked well in similar problems → successful application of NNs requires time and experience

Summary: Backpropagation algorithm 1) Uses steepest Gradient Descent (GD) for minimizing the mean square error. 2) Standard GD is slow as it requires small learning rate for stable learning. 3) GD with momentum is faster as it allows higher learning rate while maintaining stability

BackPP issues: 1) Training is slow (requires many epochs). 2) NN fully connected, too many parameters to adjust. 3) The weights are initialized randomly then adjusted by GD. 4) **Vanishing gradient problem** - the weight changes for the lower levels are very small; these layers learn slower than the higher hidden layers → less effective. 5) Require a large dataset of labeled data (sometimes not available). 6) May get stuck in local optimum or plateau. 7) Require feature-engineering, selecting useful features.

Deep learning: Networks with more than one hidden layer; Automatically learns features. Use unlabelled data for pretraining followed by supervised learning for post-training.

Autoencoder NN: - Can be used to pre-train the layers of a deep NN in advance (1 layer at a time, 1 autoencoder for each layer) - The training of a deep NN will include 3 steps:

- Pre-training step: using stacked autoencoders, train a sequence of autoencoders, 1 for each layer. Encoder compresses input x to a low-dimensional code z . Decoder reconstructs x' from z . (unsupervised).
- Fine-tuning step 1: Train the last layer using backpropagation (supervised).
- Fine-tuning step 2: train the whole network using backpropagation (supervised)

Using several autoencoders for pre-training in this way is called **stacking autoencoders**

- Each layer of the network learns an encoding of the layer below
- The network can learn hierarchical features in an unsupervised way
- The network is called a **stacked autoencoder**.

Pros: 1) 5,7 - Able to automatically learn features from unlabeled data. 2) 1,3,4,6 - Useful for pre-training layers of deep NNs (Reduces risk of poor local minima. Speeds up convergence and improves generalization.)

Purpose of pre-training: 1) Better accuracy on test data. 2) 3 - Prevent the rise in probability of landing in a non-linear minimum when add more hidden layers. 3) 4 - Provide a better starting position for the NN - in a "basin" with a better local minimum.

Convolutional Neural Networks (CNNs):

- Local connectivity: 1,2 - Designed to process image data (and other grid-like data) while drastically reducing parameters. Each hidden neuron is connected only to a small subset of inputs, corresponding to adjacent pixels, cutting down the number of weights. Each neuron is responsive to changes in its receptive field (next layer) only.
- Weight Sharing (Convolution): 1,2,7 - A filter (kernel) is applied across all positions in the input. Same weights used for each position (shared weights). Produces a **feature map**. Similar to **convolution** in signal processing where a filter (a set of weights) is applied to different positions in the input signal. The filters convolve with the image parts and "activate" (have high values) when a specific feature is detected.
- Pooling (Subsampling layer): 1,2,7 - Reduces dimensionality of feature maps. Max-pooling layer takes the maximum value of a selected set of neurons from the convolutional layer. The output of a max-pooling neuron is invariant to shifts in the inputs in its receptive field.
- Local Contrast Normalization (LCN): 5,7 - Normalizes the output of each max-pooling neuron by subtracting the μ of its incoming neurons and dividing by the σ of these neurons. Allows for **brightness invariance** → useful for image recognition.
- Dropout:** 5,6 - During training, randomly deactivate a proportion p of neurons. During testing, scale the activations by p . **Prevents overfitting** and forces the NN to be less dependent on certain neurons, to collect more evidence from other neurons → **to be more robust to noise**.

Support Vector Machines (SVMs) form decision boundaries by constructing a maximum-margin hyperplane. Can form arbitrary decision boundaries (both linear and non-linear). The decision boundary is a **maximum margin hyperplane** and helps to generalize well on new examples. The decision boundary is defined by **support vectors** → resistant to overfitting. **Margin:** Distance between the boundary and its closest examples. **Support vectors:** Data points that lie just outside the margin. Moving the support vectors can change the boundary. **Goal:** construct hyperplanes $H1$ and $H2$ such that: $H1: w \cdot x + b = 1$, $H2: w \cdot x + b = -1$, x is true iff $w \cdot x + b \geq 0$.

- To maximize the margin d of the decision boundary, need to minimize
- big margin → low complexity → lower generalization error

Optimization Problem

- During training, compute dot products between pairs of training examples;
- During testing - between the testing example and the support vectors.

λ_i are the **Lagrange multipliers** produce a **weight vector** (i.e. optimal decision boundary).

$\max \mathbf{w} \cdot \mathbf{x} = \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j=1}^N \lambda_i \lambda_j \mathbf{y}_i \mathbf{y}_j \mathbf{x}_i \cdot \mathbf{x}_j$

Target value (class value) of the training vectors

Dot product of pairs of training vectors

Original, solution

$\mathbf{w} = \sum_{i=1}^N \lambda_i \mathbf{y}_i \mathbf{x}_i$

Optimal hyperplane in the original space

subject to $\lambda_i \geq 0$, $\sum_{i=1}^N \lambda_i \mathbf{y}_i = 0$

The optimal decision boundary $\mathbf{w}$ is a linear combination (coefficient λ_i) of the "value" training vector of a small number of training examples vectors. The training examples $\mathbf{x}_i$ with non-zero λ_i are the **support vectors**, the optimal decision boundary $\mathbf{w}$ is a linear combination of support vectors.

- To classify a new example $\mathbf{z}$ → $f = \mathbf{w} \cdot \mathbf{z} + b = \sum_{i=1}^N \lambda_i \mathbf{y}_i \mathbf{x}_i \cdot \mathbf{z} + b$

Hard-Margin SVM: Ensures every point on the correct side and at least one margin unit away. **Soft-Margin SVM:** The optimization problem formulation is similar except each Lagrange multiplier satisfies $0 \leq \lambda_i \leq C$, where C controls the trade-off between maximizing the margin and allowing misclassifications (when the problem is not linearly separable, to maximize accuracy). A large C makes the classifier try harder to avoid errors (narrower margin), a small C allows more misclassifications but yields a wider margin.

Non-linear SVM: 1) It is nonlinear. 2) It is to a higher dimensional space

Issue: 1) Mapping data into a very high (even infinite) dimensional space guarantees separability in principle, but may **overfitting**. **Solution:** constructing the maximum margin hyperplane in the new space. 2) Solving QP in the new, higher dimensional space is computationally expensive. **Solution: KernelTrick:** Computing the dot product of a pair of vectors in the new space without first computing the transformation of each vector from the original to the new space. $\phi(u) \phi(v) = (u \cdot v)^2 = (K(u, v))^2$, **kernel function**, K (should satisfy **Mercer's Theorem** (be symmetric, continuous, and positive semidefinite))

$\max \mathbf{w} \cdot \mathbf{x} = \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j=1}^N \lambda_i \lambda_j \mathbf{y}_i \mathbf{y}_j K(\mathbf{x}_i, \mathbf{x}_j)$

Optimal hyperplane in the new space

$f = \mathbf{w} \cdot \mathbf{z} + b = \sum_{i=1}^N \lambda_i \mathbf{y}_i K(\mathbf{x}_i, \mathbf{z}) + b$

subject to $\lambda_i \geq 0$, $\sum_{i=1}^N \lambda_i \mathbf{y}_i = 0$

- Three key concepts:** 1) Maximize margin of the decision boundary; 2) Transform data into a higher dimensional space → more likely to be linearly separable; 3) Kernel trick - do the calculations in the original, not in the new higher dimensional space
- Linear SVM** - No kernel function; finds the **optimal linear decision boundary** between the positive and negative examples in the original feature space
- Non-linear SVM** - Uses kernel function; finds the **optimal linear boundary in the new space**. This boundary (hyperplane) when mapped back to the original feature space, corresponds to a non-linear decision boundary between the pos and neg examples.
- Multi-class problems** need to be transformed into 2-class problems

SVM, Perceptrons and Backpropagation NNs - Comparison

- Perceptrons:** simple and efficient training algorithm but can learn only linear decision boundaries. **Backpropagation NNs:** hard to train (too many parameters to set, local minima) but can learn non-linear boundaries. **SVM:** relatively efficient training algorithm that can learn non-linear decision boundaries

Ensembles are combinations of multiple base classifiers that allow for a majority voting approach. **When works well**

- 1) **Error Diversity:** classifiers make different, uncorrelated mistakes, they are independent of each other. 2) **Individual Accuracy:** each base classifier does better than random guessing ($\epsilon < 0.5$ for binary classifiers).

Methods for Creating Ensembles (generating Diversity): 1) **Manipulating Training Data:** creating multiple training sets by resampling the original data according to some sampling distribution and creating a classifier for each training set (**Bagging, Boosting**).

2) **Manipulating Features (Attributes):** Use subsets of input features (Random Forest). **Bagging (Bootstrap Aggregating):** Sample the training data with replacement to create multiple datasets. Train one classifier per dataset. Combine predictions via majority vote. **When is Useful:** Typically performs significantly better than the single classifier and is never substantially worse. Especially effective with unstable learners since small data changes lead to large prediction changes e.g. DT. Averaging predictions reduces mean-squared error in numerical prediction (regression).

Boosting: The next classifier should be created using examples that were difficult for the previous classifiers. The higher the weight, the more difficult the example was to classify by the previous classifiers. **AdaBoost (Adaptive Boosting):** Uses

- A probability for selecting the examples for the training set of the next classifier
- A weighted majority vote based on the performance of the base classifiers on the training data. **Example:**

1. Examples 2 and 9 are misclassified → their errors ϵ are 1, while the errors of the other 8 examples are 0. The weighted error of the classifier $C2$ will be: $\epsilon_2 = 0.07 + 0.17 + 1 = 0.24$

2. The term β for the probability modification will be: $\beta_2 = e^{-\epsilon_2} = e^{-0.24} \approx 0.786$

3. Calculating the new probabilities by modifying (reducing) only the probabilities of the correctly classified examples by: $p(\mathbf{x}_i) = p(\mathbf{x}_i) * \beta_2$

4. Normalization: Sum of all probabilities:

$0.07 + 0.132 + 0.108 + 0.24 = 0.55$

$0.07/0.55 = 0.127$, $0.132/0.55 = 0.24$, $0.108/0.55 = 0.196$, $0.24/0.55 = 0.436$

$p(\mathbf{x}_1) = 0.127$, $p(\mathbf{x}_2) = 0.24$, $p(\mathbf{x}_3) = 0.196$, $p(\mathbf{x}_4) = 0.436$

$p(\mathbf{x}_5) = 0.07$, $p(\mathbf{x}_6) = 0.07$, $p(\mathbf{x}_7) = 0.07$, $p(\mathbf{x}_8) = 0.07$, $p(\mathbf{x}_9) = 0.17$, $p(\mathbf{x}_{10}) = 0.17$

$p(\mathbf{x}_{11}) = 0.07$, $p(\mathbf{x}_{12}) = 0.07$, $p(\mathbf{x}_{13}) = 0.07$, $p(\mathbf{x}_{14}) = 0.07$, $p(\mathbf{x}_{15}) = 0.07$, $p(\mathbf{x}_{16}) = 0.07$, $p(\mathbf{x}_{17}) = 0.07$, $p(\mathbf{x}_{18}) = 0.07$, $p(\mathbf{x}_{19}) = 0.07$, $p(\mathbf{x}_{20}) = 0.07$

$p(\mathbf{x}_{21}) = 0.07$, $p(\mathbf{x}_{22}) = 0.07$, $p(\mathbf{x}_{23}) = 0.07$, $p(\mathbf{x}_{24}) = 0.07$, $p(\mathbf{x}_{25}) = 0.07$, $p(\mathbf{x}_{26}) = 0.07$, $p(\mathbf{x}_{27}) = 0.07$, $p(\mathbf{x}_{28}) = 0.07$, $p(\mathbf{x}_{29}) = 0.07$, $p(\mathbf{x}_{30}) = 0.07$

Random Forest: Create: Random Forest = Bagging (generate a bootstrap sample + **Random Feature Selection for selecting the best attribute** (Grow a DT without pruning. At each split, only consider k random features ($k <$ total features m). Select best using information gain.) **Classification:** Apply the new example to each of the M decision trees. Assign it to the class corresponding to the leaf. Combine the decisions of the individual trees by majority voting.

Summary: Key factors: 1) Accuracy of the individual trees (strength). 2) Correlation with each other; $\uparrow$ accuracy, $\uparrow$ correlation $\rightarrow$ $\uparrow$ performance. **Bagging+RFS:** $\uparrow$ diversity, $\downarrow$ correlation. **Feature-subset size k :** $k \rightarrow \uparrow$ strength but $\uparrow$ correlation; Rule-of-thumb: $k = \log_2(m)$. **Pros:** Outperforms a single DT. Probably resists overfitting. Faster than AdaBoost with similar accuracy

Probabilistic Reasoning. $P(A|B) = \frac{P(A, B)}{P(B)}$ if $P(B) \neq 0$ $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

- Conditional Probability:** $P(A|B) = \frac{P(A, B)}{P(B)}$ if A and B are independent, $P(A, B) = P(A|B) * P(B)$ if not.
- Bayes Theorem:** $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$
- Disjunction:** $P(A \vee B) = P(A) + P(B) - P(A, B)$
- Chain Rule:** $P(A, B, C, D) = P(A|B, C, D) * P(B|C, D) * P(C|D) * P(D)$
- Normalization:** With α (i.e. without calculating denominator)

$P(\text{cat}) \propto \alpha \cdot 0.12 \cdot 0.08 \cdot \alpha = 1 / (0.12 + 0.08) = 1 / 0.2$, $P(\text{cat}) \propto 0.12 / 0.2 = 0.6$, $P(\text{dog}) \propto 0.12 / 0.2 = 0.6$

Theorem of Total Probability: $P(A) = P(A|B)P(B) + P(A|\neg B)P(\neg B)$

Bayesian Networks Ex. What is the probability that there are lots of tourists $P(t)$?

Inference by enumeration

$R = \begin{matrix} T & F \\ L & 0.8 \\ F & 0.3 \end{matrix}$

$L = \begin{matrix} T & F \\ Q & 0.9 \\ F & 0.7 \end{matrix}$

$Q = \begin{matrix} T & F \\ R & 0.9 \\ F & 0.7 \end{matrix}$

$R = \begin{matrix} T & F \\ S & 0.7 \\ F & 0.3 \end{matrix}$

$S = \begin{matrix} T & F \\ T & 0.7 \\ F & 0.3 \end{matrix}$

$P(t) = \frac{P(t)}{0.2} = \frac{P(t)}{0.2}$

Q = T

$P(t) = \frac{P(t)}{0.2} = \frac{P(t)}{0.2}$

3. sum over S

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

4. multiply factors

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

5. sum over Q

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

6. multiply factors

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

7. keep going until sum over R

What is the probability that quokkas are happy, given there are lots of quokkas selfish being taken and it's not raining?

$P(q | s, \neg r) = \frac{P(q, s, \neg r)}{P(s, \neg r)}$

$P(q, s, \neg r) = \frac{P(\neg r)}{0.8} \sum_{i=1}^N P(t_i | q, \neg r) \sum_{j=1}^N P(s_j | q, \neg r) \sum_{k=1}^N P(L_k | \neg r) \sum_{l=1}^N P(Q_l | L_k)$

8. normalize

$P(q, s, \neg r) = \frac{P(q, s, \neg r)}{P(s, \neg r)}$

Markov blanket of a node is the node's parents, children, and children's parents

- A node is conditionally independent of all nodes outside its Markov blanket.
- A node is conditionally independent of its non-descendants given its parents.

Unsupervised Learning (Clustering) - The process of partitioning data into groups (clusters) such that items in the same cluster are: 1. similar to each other 2. dissimilar to those in other clusters.

Distance between clusters

- Centroid (means)** - middle
- Medoid** - the centrally located data point
- Single link (MIN)** - The smallest pairwise distance between elements from each cluster
- Complete link (MAX)** - the largest pairwise distance
- Average link** - the average pairwise distance

A Good Clustering will produce clusters with 1) High cohesion (High similarity within clusters). 2) High separation (Low similarity between clusters).

- Are measured with a distance function. Various ways to combine them in 1 measure: **Davies-Bouldin (DB) index:** A heuristic measure of the quality of the clustering. Combines c and s . Each pair of clusters i and j (from the resulting clustering) are compared in pairs: c : number of clusters, $c_{i,j}$ = centroid, $\text{dist}(x, c_i)$ = mean-squared distance from each item x in cluster i to its centroid. **Small DB index = clusters have small spread and are far from each other = good clustering**

$DB = \frac{1}{c} \sum_{i=1}^c \max_{j \neq i} \frac{\text{dist}(x, c_i) + \text{dist}(x, c_j)}{\text{dist}(c_i, c_j)}$

Random Forest: Create: Random Forest = Bagging (generate a bootstrap sample + **Random Feature Selection for selecting the best attribute** (Grow a DT without pruning. At each split, only consider k random features ($k <$ total features m). Select best using information gain.) **Classification:** Apply the new example to each of the M decision trees. Assign it to the class corresponding to the leaf. Combine the decisions of the individual trees by majority voting.

Summary: Key factors: 1) Accuracy of the individual trees (strength). 2) Correlation with each other; $\uparrow$ accuracy, $\uparrow$ correlation $\rightarrow$ $\uparrow$ performance. **Bagging+RFS:** $\uparrow$ diversity, $\downarrow$ correlation. **Feature-subset size k :** $k \rightarrow \uparrow$ strength but $\uparrow$ correlation; Rule-of-thumb: $k = \log_2(m)$. **Pros:** Outperforms a single DT. Probably resists overfitting. Faster than AdaBoost with similar accuracy

Probabilistic Reasoning. $P(A|B) = \frac{P(A, B)}{P(B)}$ if $P(B) \neq 0$ $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

- Conditional Probability:** $P(A|B) = \frac{P(A, B)}{P(B)}$ if A and B are independent, $P(A, B) = P(A|B) * P(B)$ if not.
- Bayes Theorem:** $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$
- Disjunction:** $P(A \vee B) = P(A) + P(B) - P(A, B)$
- Chain Rule:** $P(A, B, C, D) = P(A|B, C, D) * P(B|C, D) * P(C|D) * P(D)$
- Normalization:** With α (i.e. without calculating denominator)

$P(\text{cat}) \propto \alpha \cdot 0.12 \cdot 0.08 \cdot \alpha = 1 / (0.12 + 0.08) = 1 / 0.2$, $P(\text{cat}) \propto 0.12 / 0.2 = 0.6$, $P(\text{dog}) \propto 0.12 / 0.2 = 0.6$

Theorem of Total Probability: $P(A) = P(A|B)P(B) + P(A|\neg B)P(\neg B)$

Bayesian Networks Ex. What is the probability that there are lots of tourists $P(t)$?

Inference by enumeration

$R = \begin{matrix} T & F \\ L & 0.8 \\ F & 0.3 \end{matrix}$

$L = \begin{matrix} T & F \\ Q & 0.9 \\ F & 0.7 \end{matrix}$

$Q = \begin{matrix} T & F \\ R & 0.9 \\ F & 0.7 \end{matrix}$

$R = \begin{matrix} T & F \\ S & 0.7 \\ F & 0.3 \end{matrix}$

$S = \begin{matrix} T & F \\ T & 0.7 \\ F & 0.3 \end{matrix}$

$P(t) = \frac{P(t)}{0.2} = \frac{P(t)}{0.2}$

Q = T

$P(t) = \frac{P(t)}{0.2} = \frac{P(t)}{0.2}$

3. sum over S

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

4. multiply factors

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

5. sum over Q

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

6. multiply factors

$P(t) = \sum_{i=1}^N P(t) = \sum_{i=1}^N P(t)$

7. keep going until sum over R

What is the probability that quokkas are happy, given there are lots of quokkas selfish being taken and it's not raining?

$P(q | s, \neg r) = \frac{P(q, s, \neg r)}{P(s, \neg r)}$

$P(q, s, \neg r) = \frac{P(\neg r)}{0.8} \sum_{i=1}^N P(t_i | q, \neg r) \sum_{j=1}^N P(s_j | q, \neg r) \sum_{k=1}^N P(L_k | \neg r) \sum_{l=1}^N P(Q_l | L_k)$

8. normalize

$P(q, s, \neg r) = \frac{P(q, s, \neg r)}{P(s, \neg r)}$

Markov blanket of a node is the node's parents, children, and children's parents

- A node is conditionally independent of all nodes outside its Markov blanket.
- A node is conditionally independent of its non-descendants given its parents.

Unsupervised Learning (Clustering) - The process of partitioning data into groups (clusters) such that items in the same cluster are: 1. similar to each other 2. dissimilar to those in other clusters.

Distance between clusters

- Centroid (means)** - middle
- Medoid** - the centrally located data point
- Single link (MIN)** - The smallest pairwise distance between elements from each cluster
- Complete link (MAX)** - the largest pairwise distance
- Average link** - the average pairwise distance

A Good Clustering will produce clusters with 1) High cohesion (High similarity within clusters). 2) High separation (Low similarity between clusters).

- Are measured with a distance function. Various ways to combine them in 1 measure: **Davies-Bouldin (DB) index:** A heuristic measure of the quality of the clustering. Combines c and s . Each pair of clusters i and j (from the resulting clustering) are compared in pairs: c : number of clusters, $c_{i,j}$ = centroid, $\text{dist}(x, c_i)$ = mean-squared distance from each item x in cluster i to its centroid. **Small DB index = clusters have small spread and are far from each other = good clustering**

$DB = \frac{1}{c} \sum_{i=1}^c \max_{j \neq i} \frac{\text{dist}(x, c_i) + \text{dist}(x, c_j)}{\text{dist}(c_i, c_j)}$